Week 5: European Call Option

Qimu Luo

June 15, 2026

1 References

The reference for this week:

- FDM's theory and implementation: Chapter 18 and 19 in [2]
- PINNs:
  - DeepXDE framework in [1];
  - Documentation: `https://deepxde.readthedocs.io/en/latest/demos/pinn_forward.html`
  - Code:
    - Official Github link: `https://github.com/lululxvi/deepxde/tree/master`
    - BS Call example PR: `https://github.com/lululxvi/deepxde/pull/2056`, which adapts from the heat equation.

2 Problem Setup

We consider the pricing of a European call option under the Black–Scholes framework. Let $V(S,t)$ denote the option price, where $S > 0$ is the underlying asset price and $t \in [0,T]$ is time, the parabolic PDE is:

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS\frac{\partial V}{\partial S} - rV = 0, \quad 0 < S < S_{\max},\ 0 \le t < T. \tag{1}$$

- Terminal condition (IC): At maturity $t = T$, the option payoff is $V(S,T) = \max(S-K, 0)$.
- Boundary conditions (BC):
  - At the lower boundary: $V(0,t) = 0, \quad 0 \le t \le T$.
  - At the upper boundary, using the asymptotic behaviour of a call option, $V(S_{\max}, t) = S_{\max} - Ke^{-r(T-t)}, \quad 0 \le t \le T$.

K is the strike price, r is the risk-free interest rate, and σ is the volatility. The computational domain is truncated at a sufficiently large $S_{\max}$.

3 FDM

3.1 Fully Implicit FDM

Given that we already know the option value at maturity, the numerical solution is obtained by marching backward in time, from $t = T$ to $t = 0$. The key idea of the fully implicit method is that all spatial derivative terms are evaluated at the **unknown current time level**. This leads to a linear system that must be solved at each time step. Let the spatial interval $[0, S_{\max}]$ be divided into M equal subintervals, with

- Space: mesh size $\Delta S = \frac{S_{\max}}{M}$, and grid points $S_i = i\Delta S, \ \ i = 0, 1, \ldots, M$.
- Time: $\Delta t = \frac{T}{N}$, and time levels $t_n = n\Delta t, \ \ n = 0, 1, \ldots, N$.

The grid setting is illustrated in Figure 1.

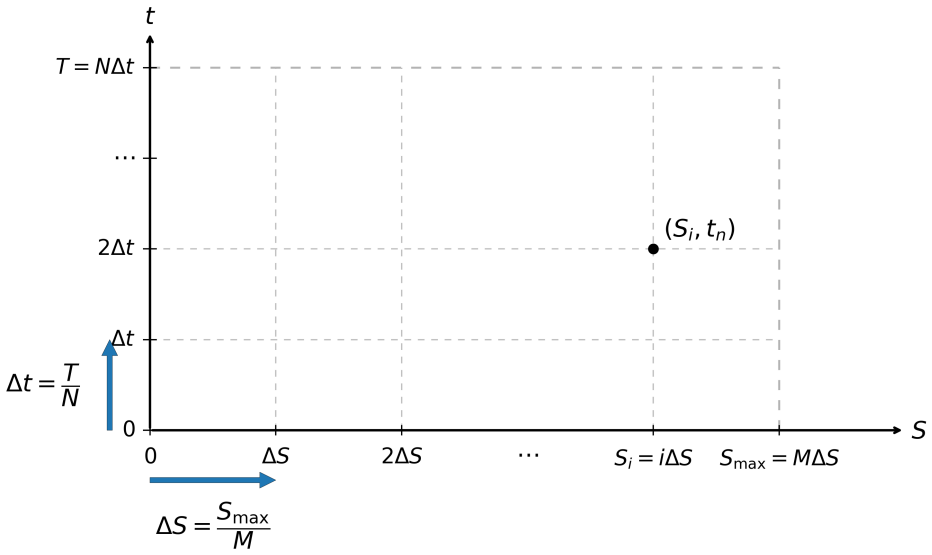


Figure 1: Illustration for Time and Spatial Variables

3.1.1 Interior Condition

Using the idea of replacing the derivative through difference scheme, the fully implicit method uses the following derivatives at the interior nodes $i = 1, 2, \ldots, M-1$:

- Time derivative: $\frac{\partial V}{\partial t}(S_i, t_n) \approx \frac{V_i^n - V_i^{n+1}}{\Delta t}$.
- First derivative in space: $\frac{\partial V}{\partial S}(S_i, t_n) \approx \frac{V_{i+1}^n - V_{i-1}^n}{2\Delta S}$.
- Second derivative in space: $\frac{\partial^2 V}{\partial S^2}(S_i, t_n) \approx \frac{V_{i+1}^n - 2V_i^n + V_{i-1}^n}{\Delta S^2}$.

Substituting these approximations into the BS equation gives

$$\frac{V_i^n - V_i^{n+1}}{\Delta t} + \frac{1}{2}\sigma^2 S_i^2 \frac{V_{i+1}^n - 2V_i^n + V_{i-1}^n}{\Delta S^2} + rS_i \frac{V_{i+1}^n - V_{i-1}^n}{2\Delta S} - rV_i^n = 0. \tag{2}$$

Multiplying through by Δt,

$$V_i^n - V_i^{n+1} + \frac{1}{2}\sigma^2 S_i^2 \Delta t \frac{V_{i+1}^n - 2V_i^n + V_{i-1}^n}{\Delta S^2} + rS_i \Delta t \frac{V_{i+1}^n - V_{i-1}^n}{2\Delta S} - r\Delta t\, V_i^n = 0. \tag{3}$$

Then, after the collection of the coefficients for V_{i-1}^n, V_i^n, and V_{i+1}^n:

$$\left(\frac{1}{2}\sigma^2 S_i^2 \frac{\Delta t}{\Delta S^2} - \frac{rS_i \Delta t}{2\Delta S}\right) V_{i-1}^n + \left(1 - \sigma^2 S_i^2 \frac{\Delta t}{\Delta S^2} - r\Delta t\right) V_i^n + \left(\frac{1}{2}\sigma^2 S_i^2 \frac{\Delta t}{\Delta S^2} + \frac{rS_i \Delta t}{2\Delta S}\right) V_{i+1}^n = V_i^{n+1}. \tag{4}$$

Since $S_i = i\Delta S$, the scheme becomes

$$\frac{1}{2}\Delta t(\sigma^2 i^2 - ri) V_{i-1}^n + \left[1 - \Delta t(\sigma^2 i^2 + r)\right] V_i^n + \frac{1}{2}\Delta t(\sigma^2 i^2 + ri) V_{i+1}^n = V_i^{n+1}. \tag{5}$$

This form is algebraically correct, but for implementation it is more convenient to move the off-diagonal terms to the left in the standard implicit form:

$$-\frac{1}{2}\Delta t(\sigma^2 i^2 - ri) V_{i-1}^n + \left[1 + \Delta t(\sigma^2 i^2 + r)\right] V_i^n - \frac{1}{2}\Delta t(\sigma^2 i^2 + ri) V_{i+1}^n = V_i^{n+1}. \tag{6}$$

We now define

$$a_i = -\frac{1}{2}\Delta t(\sigma^2 i^2 - ri), \tag{7}$$

$$b_i = 1 + \Delta t(\sigma^2 i^2 + r), \tag{8}$$

$$c_i = -\frac{1}{2}\Delta t(\sigma^2 i^2 + ri), \tag{9}$$

so that the fully implicit scheme can be written compactly as

$$a_i V_{i-1}^n + b_i V_i^n + c_i V_{i+1}^n = V_i^{n+1}, \qquad i = 1, 2, \ldots, M-1. \tag{10}$$

3.1.2 Boundary Treatment

At each time level, the boundary conditions are

$$V_0^n = 0, \qquad V_M^n = S_{\max} - Ke^{-r(T-t_n)}. \tag{11}$$

Thus, the first and last interior equations involve known boundary values.

- For $i = 1$, $b_1 V_1^n + c_1 V_2^n = V_1^{n+1} - a_1 V_0^n$.
- For $i = M-1$, $a_{M-1} V_{M-2}^n + b_{M-1} V_{M-1}^n = V_{M-1}^{n+1} - c_{M-1} V_M^n$.

3.1.3 Matrix Form

Let

$$\mathbf{V}^n = \begin{bmatrix} V_1^n \\ V_2^n \\ \vdots \\ V_{M-1}^n \end{bmatrix}, \qquad \mathbf{V}^{n+1} = \begin{bmatrix} V_1^{n+1} \\ V_2^{n+1} \\ \vdots \\ V_{M-1}^{n+1} \end{bmatrix}. \tag{12}$$

Then the fully implicit scheme can be written in matrix form as

$$A\mathbf{V}^n = \mathbf{d}^{\,n+1}, \tag{13}$$

where A is the tridiagonal matrix

$$A = \begin{bmatrix} b_1 & c_1 & 0 & \cdots & 0 \\ a_2 & b_2 & c_2 & \ddots & \vdots \\ 0 & a_3 & b_3 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & c_{M-2} \\ 0 & \cdots & 0 & a_{M-1} & b_{M-1} \end{bmatrix}, \tag{14}$$

and the right-hand side vector is

$$\mathbf{d}^{\,n+1} = \begin{bmatrix} V_1^{n+1} - a_1 V_0^n \\ V_2^{n+1} \\ \vdots \\ V_{M-2}^{n+1} \\ V_{M-1}^{n+1} - c_{M-1} V_M^n \end{bmatrix}. \tag{15}$$

Thus, at each backward time step, one solves a tridiagonal linear system for the interior values $\mathbf{V}^n$.

3.1.4 Code Implementation

The implementation can be summarised in the following key steps:

1. Set up grids and initialize the terminal condition. And form the coefficient vectors. For interior nodes $i = 1, \ldots, M-1$, compute

$$a_i = -\frac{1}{2}\Delta t(\sigma^2 i^2 - ri), \quad b_i = 1 + \Delta t(\sigma^2 i^2 + r), \quad c_i = -\frac{1}{2}\Delta t(\sigma^2 i^2 + ri),$$

corresponding to the tridiagonal system.

2. Construct the banded matrix: Store the tridiagonal system efficiently by keeping only the three diagonals. For example, when $M = 5$,

$$\texttt{ab} = \begin{bmatrix} 0 & c_1 & c_2 & c_3 \\ b_1 & b_2 & b_3 & b_4 \\ a_2 & a_3 & a_4 & 0 \end{bmatrix},$$

where the rows represent the upper, main, and lower diagonals, respectively.

3. **Backward time stepping.** For each time step:
   - impose boundary conditions V_0^n and V_M^n;
   - use the known values at level $n+1$ to form the right-hand side;
   - adjust the first and last entries to account for boundary terms (as described in section 3.1.2);
   - solve the tridiagonal system for the interior values.
4. Reconstruct the full solution vector at each step and repeat until $t = 0$. The final result is then compared with the analytical BS solution to evaluate accuracy.

3.2 Explicit FDM

The explicit finite difference method uses the same spatial grid and boundary conditions as the fully implicit method, but the key difference is that all spatial derivative terms are evaluated at the **known next time level**. Thus, the scheme directly updates the solution from V^{n+1} to V^n without solving a linear system. For the fully implicit method, the discretization gives

$$a_i V_{i-1}^n + b_i V_i^n + c_i V_{i+1}^n = V_i^{n+1}, \tag{16}$$

so the **unknown** values at time level n are coupled together, and a tridiagonal linear system must be solved at each step. By contrast, the explicit method evaluates the spatial terms at the known level $n+1$. This gives

$$V_i^n = a_i V_{i-1}^{n+1} + b_i V_i^{n+1} + c_i V_{i+1}^{n+1}, \tag{17}$$

where

$$\begin{aligned} a_i &= \frac{1}{2}\Delta t(\sigma^2 i^2 - ri), \\ b_i &= 1 - \Delta t(\sigma^2 i^2 + r), \\ c_i &= \frac{1}{2}\Delta t(\sigma^2 i^2 + ri). \end{aligned}$$

Therefore, unlike the fully implicit method, the explicit method does not produce a tridiagonal system. Each interior value V_i^n can be computed directly from the already known values at the next time level. (Note for $b_i \geq 0$). The code implementation can be summarised as follows:

1. Set up the grids and initialize the terminal condition and construct the explicit coefficients.
2. Check the stability condition: Since the explicit method is conditionally stable, the code checks whether Δt is sufficiently small relative to the spatial grid.
3. March backward in time explicitly.
4. Repeat until $t = 0$ and compare with the exact solution.

3.3 Implicit FDM (Crank–Nicolson)

The CN method can be viewed as a refinement of the fully implicit method. Its key idea is to evaluate the spatial operator at the average of two consecutive time levels, rather than using only the unknown current level or only the known next level. Hence, it may be regarded as the average of the implicit and explicit time discretizations. Let L_h denote the discrete spatial operator. Then the CN scheme is

$$\frac{V_i^n - V_i^{n+1}}{\Delta t} = \frac{1}{2}\Big(L_h V_i^n + L_h V_i^{n+1}\Big). \tag{18}$$

Using the same spatial discretization as in the fully implicit method,

$$L_h V_i = \frac{1}{2}(\sigma^2 i^2 - ri)V_{i-1} - (\sigma^2 i^2 + r)V_i + \frac{1}{2}(\sigma^2 i^2 + ri)V_{i+1}, \tag{19}$$

the Crank–Nicolson scheme becomes

$$\left(I - \frac{\Delta t}{2} L_h\right) V^n = \left(I + \frac{\Delta t}{2} L_h\right) V^{n+1}. \tag{20}$$

Equivalently, for each interior node,

$$a_i^{(L)} V_{i-1}^n + b_i^{(L)} V_i^n + c_i^{(L)} V_{i+1}^n = a_i^{(R)} V_{i-1}^{n+1} + b_i^{(R)} V_i^{n+1} + c_i^{(R)} V_{i+1}^{n+1}, \tag{21}$$

where

$$\begin{aligned} a_i^{(L)} &= -\frac{1}{4}\Delta t(\sigma^2 i^2 - ri), \\ b_i^{(L)} &= 1 + \frac{1}{2}\Delta t(\sigma^2 i^2 + r), \\ c_i^{(L)} &= -\frac{1}{4}\Delta t(\sigma^2 i^2 + ri), \end{aligned}$$

and

$$\begin{aligned} a_i^{(R)} &= \frac{1}{4}\Delta t(\sigma^2 i^2 - ri), \\ b_i^{(R)} &= 1 - \frac{1}{2}\Delta t(\sigma^2 i^2 + r), \\ c_i^{(R)} &= \frac{1}{4}\Delta t(\sigma^2 i^2 + ri). \end{aligned}$$

Thus, as in the fully implicit method, one still solves a tridiagonal linear system at each time step, but the right-hand side now also contains contributions from the neighboring nodes at the known level $n + 1$. The implementation can be summarised as follows:

1. Set up the spatial and time grids and initialize the terminal condition.
2. Construct the discrete Black–Scholes operator. For the interior nodes $i = 1, \ldots, M - 1$, compute the coefficients of the spatial operator and then form the LHS and RHS coefficient vectors.
3. Build the banded matrix for the left-hand side. As in the fully implicit method, only the three diagonals of the tridiagonal matrix are stored in banded form for efficient solution.
4. March backward in time. At each time step:
   - compute the right-hand side using the known solution at level $n + 1$;
   - impose the boundary values at both t_n and t_{n+1};
   - adjust the first and last equations for the boundary terms;
   - solve the resulting tridiagonal system for the interior values at level n.
5. Update and verify the solution.

4 θ method: General form

5 PINN (DeepXDE Framework)

Instead of discretizing the PDE on a finite difference grid, the solution is approximated by a neural network $V_\theta(S, \tau)$, where θ denotes the trainable parameters. The implementation can be summarised as follows:

1. Define the PDE residual. The BS equation in time-to-maturity form is written as

$$\frac{\partial V}{\partial \tau} - \left(\frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS\frac{\partial V}{\partial S} - rV \right) = 0. \tag{22}$$

   In DeepXDE, automatic differentiation is used to compute the derivatives V_τ, V_S, and V_{SS}, and the PDE residual is enforced directly in the loss function.

2. Define the computational domain and constraints. The problem is posed on the space–time domain

$$(S, \tau) \in [0, S_{\max}] \times [0, T].$$

   The terminal payoff becomes the initial condition in τ:

$$V(S, 0) = \max(S - K, 0). \tag{23}$$

   The boundary conditions are

$$V(0, \tau) = 0, \qquad V(S_{\max}, \tau) = S_{\max} - Ke^{-r\tau}. \tag{24}$$

3. Construct the DeepXDE data object. The interior collocation points, boundary points, and initial points are sampled in the space–time domain, and the PDE residual together with the boundary and initial conditions are assembled into a single training problem.

4. Build and train the neural network. A fully connected neural network with input (S, τ) and output $V_\theta(S, \tau)$ is used. The model is first trained with the Adam optimizer and then refined by L-BFGS to improve accuracy.

5. Validate against the analytical solution.

6 Results

The BS problem parameters were set as

- $K = 100.0$
- $r = 0.05$
- $\sigma = 0.2$
- $T = 1.0$
- $S_{\max} = 300.0$

6.1 Comparison of FDM schemes for fixed parameters

The comparison includes:

1. Original case: Fixed $(M, N,)$, σ: in Table 1.
2. Comparison cases: Vary (M, N); Time refinement; Space refinement; Vary σ
   - Explicit case in Table 2
   - Implicit case in Table 3
   - CN case in Table 4

Note that for the original case, the explicit method need a small Δt for stable condition, thus, by setting $N = 1.1 * \lceil T/\Delta t_{\text{limit}} \rceil = 7006$, I obtain the same N in the implicit and Crank–Nicolson schemes to ensure a fair comparison across methods in Table 1. The figure comparison for each method's

- overall error: Figure 2.
- stability in reaction to σ: Figure 3.

Table 1: Comparison of Explicit, Implicit, and Crank–Nicolson Methods (Original Case)

Method	M	N	σ	Max Error	RMSE	Relative L^2 Error	Pointwise Error
Explicit	400	7006	0.2000	3.37×10^{-4}	9.74×10^{-5}	9.94×10^{-7}	6.91×10^{-5}
Implicit	400	7006	0.2000	5.53×10^{-4}	1.60×10^{-4}	1.63×10^{-6}	3.67×10^{-4}
Crank–Nicolson	400	7006	0.2000	4.45×10^{-4}	1.23×10^{-4}	1.26×10^{-6}	2.18×10^{-4}

Table 2: Explicit Method: Summary of All Comparisons

Case	(M, N)	σ	Max Error	RMSE	Relative L^2 Error	Pointwise Error
Comparison 1: Coupled refinement (stability-constrained)						
	(50,250)	0.20	2.67×10^{-2}	7.16×10^{-3}	7.23×10^{-5}	8.97×10^{-3}
	(100,1000)	0.20	6.20×10^{-3}	1.75×10^{-3}	1.78×10^{-5}	2.54×10^{-3}
	(200,4000)	0.20	1.62×10^{-3}	4.43×10^{-4}	4.52×10^{-6}	5.92×10^{-4}
	(400,16000)	0.20	3.98×10^{-4}	1.10×10^{-4}	1.13×10^{-6}	1.53×10^{-4}
Comparison 2: Fix $M = 200$ (time refinement)						
	(200,1000)	0.20	1.02×10^{289}	∞	∞	2.37×10^{187}
	(200,2000)	0.20	1.43×10^{-3}	4.01×10^{-4}	4.08×10^{-6}	3.28×10^{-4}
	(200,4000)	0.20	1.62×10^{-3}	4.43×10^{-4}	4.52×10^{-6}	5.92×10^{-4}
	(200,8000)	0.20	1.71×10^{-3}	4.69×10^{-4}	4.78×10^{-6}	7.24×10^{-4}
	(200,16000)	0.20	1.76×10^{-3}	4.82×10^{-4}	4.92×10^{-6}	7.90×10^{-4}
Comparison 3: Fix $N = 4000$ (space refinement)						
	(50,4000)	0.20	2.95×10^{-2}	7.95×10^{-3}	8.03×10^{-5}	1.30×10^{-2}
	(100,4000)	0.20	6.74×10^{-3}	1.90×10^{-3}	1.93×10^{-5}	3.31×10^{-3}
	(200,4000)	0.20	1.62×10^{-3}	4.43×10^{-4}	4.52×10^{-6}	5.92×10^{-4}
	(400,4000)	0.20	`nan`	`nan`	`nan`	`nan`
Comparison 4: Vary σ (fixed grid 200×4000)						
	(200,4000)	0.10	5.86×10^{-3}	1.23×10^{-3}	1.26×10^{-5}	3.43×10^{-3}
	(200,4000)	0.15	2.79×10^{-3}	6.61×10^{-4}	6.74×10^{-6}	1.31×10^{-3}
	(200,4000)	0.20	1.62×10^{-3}	4.43×10^{-4}	4.52×10^{-6}	5.92×10^{-4}
	(200,4000)	0.25	1.03×10^{-3}	3.59×10^{-4}	3.66×10^{-6}	2.28×10^{-4}
	(200,4000)	0.30	7.56×10^{-4}	3.68×10^{-4}	3.74×10^{-6}	8.02×10^{-6}

6.2 PINN

The PINN experiment was implemented in DeepXDE using the `tensorflow.compat.v1` backend.

The computational domain was

$$(S, \tau) \in [0, S_{\max}] \times [0, T].$$

The training data in DeepXDE were specified as follows:

- interior collocation points: `num_domain = 2047`
- boundary points: `num_boundary = 63`
- initial points: `num_initial = 127`
- test points: `num_test = 2047`
- training distribution: `Sobol`

The neural network architecture was

$$[2] + [28] \times 4 + [1],$$

that is, a fully connected neural network with 2 inputs, 4 hidden layers of width 28, and 1 output, using `tanh` activation and `Glorot normal` initialization. Training was carried out in two stages:

Table 3: Implicit Method: Summary of All Comparisons

Case	(M, N)	σ	Max Error	RMSE	Relative L^2 Error	Pointwise Error
Comparison 1: $M = N$ **(grid refinement)**						
	(50,50)	0.20	4.43×10^{-2}	1.34×10^{-2}	1.35×10^{-4}	3.47×10^{-2}
	(100,100)	0.20	1.44×10^{-2}	4.85×10^{-3}	4.93×10^{-5}	1.38×10^{-2}
	(200,200)	0.20	6.17×10^{-3}	2.04×10^{-3}	2.08×10^{-5}	6.14×10^{-3}
	(400,400)	0.20	2.88×10^{-3}	9.26×10^{-4}	9.45×10^{-6}	2.83×10^{-3}
	(800,800)	0.20	1.39×10^{-3}	4.41×10^{-4}	4.50×10^{-6}	1.37×10^{-3}
Comparison 2: Fix $M = 200$ **(time refinement)**						
	(200,50)	0.20	2.22×10^{-2}	7.00×10^{-3}	7.14×10^{-5}	2.20×10^{-2}
	(200,100)	0.20	1.15×10^{-2}	3.69×10^{-3}	3.77×10^{-5}	1.14×10^{-2}
	(200,200)	0.20	6.17×10^{-3}	2.04×10^{-3}	2.08×10^{-5}	6.14×10^{-3}
	(200,400)	0.20	3.69×10^{-3}	1.23×10^{-3}	1.25×10^{-5}	3.50×10^{-3}
	(200,800)	0.20	2.75×10^{-3}	8.40×10^{-4}	8.57×10^{-6}	2.18×10^{-3}
Comparison 3: Fix $N = 200$ **(space refinement)**						
	(50,200)	0.20	3.34×10^{-2}	9.22×10^{-3}	9.32×10^{-5}	1.86×10^{-2}
	(100,200)	0.20	1.07×10^{-2}	3.31×10^{-3}	3.36×10^{-5}	8.70×10^{-3}
	(200,200)	0.20	6.17×10^{-3}	2.04×10^{-3}	2.08×10^{-5}	6.14×10^{-3}
	(400,200)	0.20	5.56×10^{-3}	1.76×10^{-3}	1.80×10^{-5}	5.44×10^{-3}
	(800,200)	0.20	5.41×10^{-3}	1.70×10^{-3}	1.73×10^{-5}	5.31×10^{-3}
Comparison 4: Vary σ **(fixed grid** 400×400**)**						
	(400,400)	0.10	3.46×10^{-3}	7.68×10^{-4}	7.84×10^{-6}	1.95×10^{-3}
	(400,400)	0.15	2.61×10^{-3}	7.71×10^{-4}	7.88×10^{-6}	2.25×10^{-3}
	(400,400)	0.20	2.88×10^{-3}	9.26×10^{-4}	9.45×10^{-6}	2.83×10^{-3}
	(400,400)	0.25	3.52×10^{-3}	1.17×10^{-3}	1.19×10^{-5}	3.47×10^{-3}
	(400,400)	0.30	4.20×10^{-3}	1.50×10^{-3}	1.52×10^{-5}	4.14×10^{-3}

1. Adam optimizer with learning rate `1e-3` for `20000` iterations;
2. L-BFGS refinement.

The final validation results on the test set were

$$\text{Mean residual} = 2.21 \times 10^{-1},$$
$$\text{Relative } L^2 \text{ error} = 1.68 \times 10^{-3}.$$

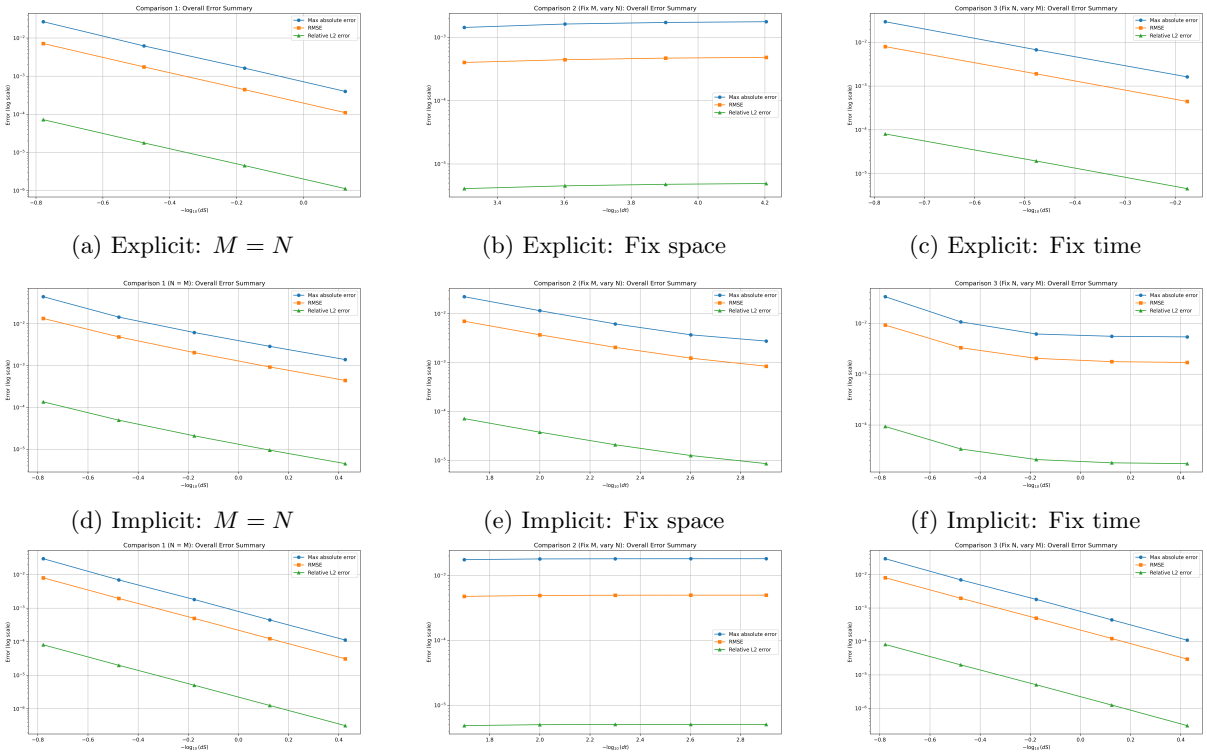
References

[1] Lu Lu, Xuhui Meng, Zhiping Mao, and George Em Karniadakis. DeepXDE: A deep learning library for solving differential equations. *SIAM Review*, 63(1):208–228, 2021.

[2] P. Wilmott, J. Dewynne, and S. Howison. *Option Pricing: Mathematical Models and Computation*. Oxford Financial Press, 1993.

Table 4: Crank–Nicolson Method: Summary of All Comparisons

Case	(M, N)	σ	Max Error	RMSE	Relative L^2 Error	Pointwise Error
Comparison 1: $M = N$ **(grid refinement)**						
	(50,50)	0.20	2.96×10^{-2}	7.99×10^{-3}	8.07×10^{-5}	1.32×10^{-2}
	(100,100)	0.20	6.92×10^{-3}	1.95×10^{-3}	1.98×10^{-5}	3.54×10^{-3}
	(200,200)	0.20	1.81×10^{-3}	4.95×10^{-4}	5.05×10^{-6}	8.50×10^{-4}
	(400,400)	0.20	4.44×10^{-4}	1.23×10^{-4}	1.26×10^{-6}	2.17×10^{-4}
	(800,800)	0.20	1.12×10^{-4}	3.09×10^{-5}	3.16×10^{-7}	5.36×10^{-5}
Comparison 2: Fix $M = 200$ **(time refinement)**						
	(200,50)	0.20	1.75×10^{-3}	4.75×10^{-4}	4.84×10^{-6}	7.56×10^{-4}
	(200,100)	0.20	1.79×10^{-3}	4.91×10^{-4}	5.01×10^{-6}	8.32×10^{-4}
	(200,200)	0.20	1.81×10^{-3}	4.95×10^{-4}	5.05×10^{-6}	8.50×10^{-4}
	(200,400)	0.20	1.81×10^{-3}	4.96×10^{-4}	5.06×10^{-6}	8.55×10^{-4}
	(200,800)	0.20	1.81×10^{-3}	4.97×10^{-4}	5.06×10^{-6}	8.56×10^{-4}
Comparison 3: Fix $N = 200$ **(space refinement)**						
	(50,200)	0.20	2.97×10^{-2}	8.01×10^{-3}	8.09×10^{-5}	1.33×10^{-2}
	(100,200)	0.20	6.92×10^{-3}	1.95×10^{-3}	1.99×10^{-5}	3.56×10^{-3}
	(200,200)	0.20	1.81×10^{-3}	4.95×10^{-4}	5.05×10^{-6}	8.50×10^{-4}
	(400,200)	0.20	4.41×10^{-4}	1.22×10^{-4}	1.25×10^{-6}	2.12×10^{-4}
	(800,200)	0.20	1.09×10^{-4}	2.96×10^{-5}	3.03×10^{-7}	4.78×10^{-5}
Comparison 4: Vary σ **(fixed grid** 400×400**)**						
	(400,400)	0.10	1.48×10^{-3}	3.11×10^{-4}	3.18×10^{-6}	8.57×10^{-4}
	(400,400)	0.15	7.30×10^{-4}	1.75×10^{-4}	1.79×10^{-6}	3.72×10^{-4}
	(400,400)	0.20	4.44×10^{-4}	1.23×10^{-4}	1.26×10^{-6}	2.17×10^{-4}
	(400,400)	0.25	3.01×10^{-4}	1.02×10^{-4}	1.04×10^{-6}	1.45×10^{-4}
	(400,400)	0.30	7.56×10^{-4}	1.69×10^{-4}	1.73×10^{-6}	1.03×10^{-4}

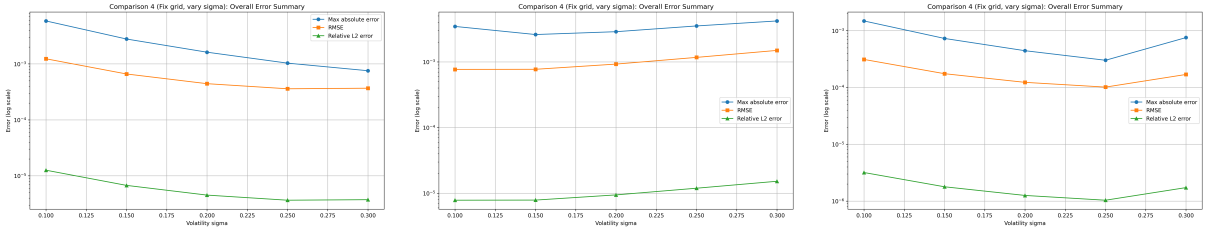


(a) Explicit: $M = N$ (b) Explicit: Fix space (c) Explicit: Fix time

(d) Implicit: $M = N$ (e) Implicit: Fix space (f) Implicit: Fix time

(g) CN: $M = N$ (h) CN: Fix space (i) CN: Fix time

Figure 2: Overall error comparison of FDMs under different grid configurations.



(a) Explicit (b) Implicit (c) Crank–Nicolson

Figure 3: Impact of volatility σ on overall error for different FDM schemes